VOID CTF
Detailed Challenge Write-ups

Evidence-based analysis of the supplied screenshots and challenge artifacts

Accuracy note. This report distinguishes verified flags from challenges for which the supplied evidence is incomplete. No flag or exploit has been invented for a challenge that could not be independently verified. Four challenge results are supported by the uploaded files; three are documented as pending verification.

Verified results at a glance

• **1. The Proof-Carrying Exploit — Pending verification.** PWN • 250 points • connection shown in challenge: 0.cloud.chals.io:1337

• **2. What Next? — Pending verification.** Web Exploitation • 200 points • https://void-ctf-whatnext.chals.io/

• **3. The Last Relay — Pending verification.** Challenge category/points shown: Misc • 100 points • https://void-ctf-the-last-relay.chals.io/

• **4. A Sealed Record — Verified.** Forensics • 300 points • flag format shown: VOID{...}

• **5. I Heard something — Verified.** Forensics • 100 points • flag format shown: VOID{...}

• **6. The last broadcast — Verified.** Forensics • 300 points • example flag format: CTF{old_mill_bridge}

• **7. The Margin Machine — Verified.** Cryptography • 350 points • flag format shown: VOID{...}

1. The Proof-Carrying Exploit

PWN • 250 points • connection shown in challenge: 0.cloud.chals.io:1337

Status: Static handout reviewed; final exploit and flag not verified.

The supplied handout.rar was extracted and inspected. It contains client_template.py, common.h, gen_proof.py, and the proof_vm executable. These files establish that the challenge uses a custom bytecode virtual machine called PCVM (SafeScript PCVM). The binary payload begins with a 16-byte header, followed by proof-rule records and 8-byte instructions. The supplied builder defines operations for allocation, pinning, deriving interior pointers, loading/storing words, forcing garbage collection, emitting a hash, and halting.

The important security boundary is the verifier. A proof certificate of type PINNED_STABILITY claims that an interior pointer remains valid across a garbage-collection interval because its parent object is pinned. The challenge statement explicitly points to a possible soundness gap in formal verification and a physical memory desynchronization during compacting GC. That suggests the intended exploit must make the verifier accept a proof whose assumptions do not remain true at runtime, then cause a stale interior pointer to be used after compaction. The instance secret is not directly printed; the VM exposes an emit-hash operation that feeds a value into an instance-specific HMAC, so the intended recovery likely requires a carefully chosen memory-read/secret-extraction primitive rather than simply printing a buffer.

Reproducible investigation steps:
1. Extract the archive and inspect the supplied format definitions in common.h and gen_proof.py.
2. Map each instruction to its byte encoding; note that instructions are 8 bytes and proof rules are 12 bytes.
3. Read the verifier and runtime separately. Record precisely which register types and proof ranges the verifier accepts, then compare those assumptions with the runtime behavior of pin/unpin, derive_ptr, and gc.
4. Build a minimal test program with ProgramBuilder and run it against the challenge service. Increase complexity only after the verifier accepts the baseline.
5. Validate any candidate proof against the actual server response and confirm the returned flag format.

Why no flag is included: the archive provides the client and local VM, but the screenshot’s instance-specific service is separate. I have not verified a working exploit against that service, so inventing a payload or flag would not be a genuine write-up.

2. What Next?

Web Exploitation • 200 points • https://void-ctf-whatnext.chals.io/

Status: Challenge identified; endpoint behavior and flag not verified.

The challenge page says that users keep opening tickets without resolution and asks why the administrator is “being so lazy.” That wording suggests the intended target is a ticket workflow: ticket creation, ticket ordering or state, and a privileged administrative action. The screenshot alone does not prove a specific vulnerability, so it would be misleading to label it as IDOR, SQL injection, or an authorization bypass without inspecting requests.

A sound investigation would proceed as follows:
1. Open browser developer tools and capture the page’s network requests while listing, creating, viewing, and updating a ticket.
2. Inspect the JavaScript bundles and API responses for route names, object identifiers, status fields, and any admin-only workflow.
3. Compare requests made by a normal user with any documented administrative interface. Test only the challenge instance and use the smallest possible changes.
4. Check whether the server enforces authorization and valid state transitions, rather than trusting hidden buttons or client-side fields.
5. Record the exact request and response that demonstrates the flaw, then retrieve the flag from the resulting server-side behavior.

The challenge host was not reachable from this analysis environment, and no source bundle, HAR capture, or challenge archive was supplied. Therefore no exploit chain or flag can be responsibly claimed yet.

3. The Last Relay

Challenge category/points shown: Misc • 100 points • https://void-ctf-the-last-relay.chals.io/

Status: Challenge identified; the required live-room evidence and session flag are not present in the supplied files.

The challenge description says the anonymous emergency IRC relay has conflicting accounts in the archive, live relay, and diagnostic client. It specifically requires reconstructing the evacuation and collecting five authoritative pieces of evidence before submitting a generated session flag. It also warns that participants may bluff and that messages/artifacts should be corroborated.

A valid write-up must preserve the evidence chain instead of treating any single message as authoritative:
1. Connect to the challenge room and record the initial state and available public services.
2. Inspect the archive, live relay, and diagnostic client independently.
3. List the five required evidence items and record the source that supports each one.
4. Compare conflicting claims; accept a claim only when it is supported by the designated authoritative evidence.
5. Submit the five-item reconstruction to the verifier and record the exact generated session flag.

The currently supplied Alder Run/Northstar FM text files belong to the separate “The last broadcast” forensics challenge; they do not establish the five live-relay evidence items. The live service was not reachable from this environment, so I have not guessed the session flag.

4. A Sealed Record

Forensics • 300 points • flag format shown: VOID{...}

Verified flag: VOID{crc_r3v34ls_th3_tru3_h31ght}

Evidence supplied: Artifact-BC34 (1).rar, containing Artifact-BC34.png. The PNG initially fails to open normally because the IHDR header has a CRC mismatch. The challenge note says the tag was rewritten after sealing and instructs the analyst to trust the seal, not the tag. In a PNG, IHDR stores the image width, height, bit depth, color type, compression method, filter method, and interlace method. The CRC covers the chunk type plus its data.

Investigation:
1. Extract the PNG from the RAR archive and walk the PNG chunks.
2. Verify that the IDAT stream decompresses successfully. Only the IHDR CRC is inconsistent; the compressed image data itself is intact.
3. Read the declared dimensions: width 2464 and suspicious height 1607. The image uses 8-bit RGB, so each scanline contains one filter byte plus 3 × 2464 pixel bytes.
4. Calculate bytes per row: 1 + (3 × 2464) = 7393.
5. Decompress the IDAT data and measure 12,775,104 bytes. Dividing by 7393 gives exactly 1728 scanlines.
6. Reconstruct the IHDR height as 1728 and recompute the CRC over the IHDR type and 13-byte data. The resulting CRC is 0x14c4f49f, matching the CRC stored in the original file.
7. Repairing the header to width 2464, height 1728 restores the image. The recovered-record strip embedded at the bottom of the image contains the flag.

Why this is reliable: the height is not guessed from the picture. It is independently derived from the decompressed scanline length and confirmed by the original IHDR CRC.

5. I Heard something

Forensics • 100 points • flag format shown: VOID{...}

Verified flag: VOID{y0u_c4n7_h34r_m3}

Evidence supplied: whisper (1).wav, a 12-second, mono, 44.1 kHz PCM WAV recording. The challenge text says the file is “just a tune” and hints that some content is meant for machines to hear. Listening alone is not sufficient: the relevant information is encoded in the high-frequency part of the recording.

Investigation:
1. Read the WAV metadata and confirm the duration and sample rate.
2. Generate a spectrogram rather than relying only on playback.
3. Inspect the upper frequency band, particularly around 16–20 kHz. The spectrogram reveals a sequence of letter- and digit-shaped patterns that are not obvious in the audible melody.
4. Read the hidden text as `VOID{y0u_c4n7_h34r_m3}`. The leetspeak phrase means “you can’t hear me.”
5. Confirm the recovered text follows the challenge’s stated VOID{...} flag format.

This solution is supported by the supplied audio itself: the flag appears as a spectrogram image in the high-frequency band, not as ordinary spoken words.

6. The last broadcast

Forensics • 300 points • example flag format: CTF{old_mill_bridge}

Verified flag: CTF{lantern_reed_bridge}

Evidence supplied: 01_station_archive.txt, 02_town_bulletin.txt, 03_trail_maintenance.txt, 04_photo_caption.txt, and 05_works_register.txt. These files describe the fictional Alder Run trail and Northstar FM relay hut. The task asks for the three-word name of the footbridge nearest the relay hut, with words lowercased and separated by underscores.

Investigation:
1. Use the town bulletin to establish the correct route. Start at the east gate and follow the crescent marker down the lower path; the sun marker leads to Quarry Rise, which is explicitly outside the public loop.
2. Do not trust the old visitor map for bridge names: it labels all three crossings only as “Footbridge.”
3. Use the volunteer log to establish physical order when walking downhill from the east gate: AR-14, then AR-09, then AR-22.
4. The same log says the relay-hut path leaves the main trail immediately after the second crossing and warns not to cross the third bridge. Therefore, the relevant crossing is AR-09.
5. Use the works register to map inventory identifier AR-09 to “Lantern Reed Bridge.” The register is sorted by maintenance date, not by trail order, so its row order must not be mistaken for the route order.
6. The archive and photo caption corroborate the route: the hut is beyond a bridge on the crescent path, and the caption describes a brass plaque near the reeds. The AR-09 register entry explicitly says it has a brass plaque and the dedication “for those who keep a light.”
7. Convert the bridge name to the requested lowercase underscore form: `lantern_reed_bridge`.

The flag is therefore CTF{lantern_reed_bridge}. The conclusion comes from combining route order with the separate asset register, not from guessing from a bridge name.

7. The Margin Machine

Cryptography • 350 points • flag format shown: VOID{...}

Verified recovered message/flag: VOID{READ_THE_MARGIN_TWICE}

Evidence supplied: machine_card.txt and operator_log.txt. The current card claims that A=0 through Z=25 and that the key LANTERN is added to each message letter modulo 26. However, the handwritten margin says the card is old and that “the wheel turns BACK now.” The examples also contradict the current card’s addition rule.

Investigation:
1. Treat the alphabet as numbers 0–25 and repeat the key LANTERN over alphabetic characters. Punctuation and underscores remain unchanged.
2. Test the first example. ATTACK AT DAWN becomes PTGHYT NI DNDJ when the key values are subtracted modulo 26. This confirms the “BACK” direction: C = (P − K) mod 26.
3. The second sample, MEET AT NOON → BERA WC ADOA, follows the same subtraction rule. Two independent examples agree, so the rule is better supported than the typed card’s addition instruction.
4. Apply the inverse rule to the encoded sample `GOVW{VVNO_TUX_QRERIA_MAZPP}`. For each letter, calculate plaintext = (ciphertext − key letter) mod 26, advancing the key only for letters.
5. The decoded text is `VOID{READ_THE_MARGIN_TWICE}`.

The key lesson is to treat the card as potentially stale and resolve conflicting instructions by testing them against the supplied known-plaintext examples. The margin note correctly signals the subtraction direction.

Evidence and limitations

The following items were directly examined: handout.rar (PCVM client, header/bytecode definitions, proof builder, and VM executable); Artifact-BC34 (1).rar (the PNG recovered from the archive); whisper (1).wav; machine_card.txt; operator_log.txt; and the five Alder Run/Northstar FM text files.

The web challenge hosts were not reachable from this analysis environment. No browser HAR capture or source files were provided for What Next?, and no live-room transcripts or verifier responses were provided for The Last Relay. The PWN archive supports static analysis of the virtual machine, but does not itself prove a working remote exploit or provide a verified instance flag.

Recommended next uploads to complete the remaining write-ups: a saved page/source or HAR capture for What Next?; the public-room transcript, diagnostic output, and verifier feedback for The Last Relay; and a successful service interaction transcript for The Proof-Carrying Exploit.